

Ethan Small

4/28/2026

CMPSC 132

Final Report

For my final project I chose to create the classic tic-tac-toe game, playable from the terminal. It's a two-player game where you and someone else take turns filling a grid and getting, usually 3, in a row. This could be vertically, horizontally, or diagonal. If every player on the board hasn't won and every grid is filled, then the game ends in a draw.

For the game I decided to create one class, Board, that holds all the primitive game logic. This class holds all the game's functions like `who_won` which returns the player id of who got 3 in a row, and also `did_win` which returns a boolean value if someone did win. It also holds the board's 2d array along with functions to create a list of n size and adding `player_id` to individual cells. Then for input I have the Board class created in a ToeGame class that is created in the main runtime, that has a function called `get_player_input` which requests an integer and repeatedly calls itself until the user enters in a valid number. The ToeGame checks for player wins after each player takes turns, because then it can catch errors where two players have won by the end of a round.

After implementing the main game logic and making it playable, I decided to do something cool for visually displaying the board. I thought about drawing out x's and o's instead of just printing an x and o's with some spacing. One way that I could have done this was use the ascii block character (█) that fills up the entire space of a character. I made a class called BoardRenderer which handles all the drawing of the board. In another file I made a base class called Renderables, which is a class that has one function called `render` that returns a str. Then I

made child classes, HorizontalLine and Vertical Line, which build the str in render. For all the pixels being printed to the screen I used the escape code that lets the program move the cursor in the terminal to an (x, y) location. This is so I could move the board to any location within the terminal itself. Then I made Renderables X and O that have the logic for generating the pixels based off of some parameters such as the square size that each cell is given. I then added color to the pixels by changing the fore color of each pixel to an rgb value. This ended up with having a vibrant colorful board being shown every turn so as to give an easier understanding of the current game state.

Some challenges that I faced while creating this project were mostly stemming from rendering the board. This included things like getting line spacing accurate, making sure variables were used and referenced properly. The circle drawing logic was very difficult for me and making sure the logic was working and that pixels were placed accurately within the square and not clipping out.

Some further improvements that I could make are making the pixels be squares instead of rectangles. The ascii block character is longer than it is wide. This means that for the grid and the x's and o's, that anything I print out will look longer and more rectangular, not a perfect square. Some way that I could fix it is using the ascii top half block (■), I could change the fore color to one color and then the back color to the another, and can represent two pixels in half the space. Another thing I could do is more in the way of refactoring the code, and instead of using boardRenderer I could make a screen class that uses the `__setitem__(self, key: tuple, value: tuple)` that takes in the x and y in a tuple and then a rgb color value and would move the cursor and print that pixel at the x and y location. This would end up making much cleaner code by just using functions to draw lines instead of making multiple classes and save time. Also just another

improvement that implementing the previous idea would do is make it easier to draw a line through the winning row or draw some type of overlay saying that they lost and so on.